

JWT omar

Explicacion completa de JWT y Autenticacion

World Cup Predictor - Spring Boot + Vanilla JS

1. Que es JWT y por que lo usamos

JWT (JSON Web Token) es un estandar (RFC 7519) que permite enviar informacion de forma segura entre dos partes como texto firmado digitalmente. En este proyecto lo usamos para autenticacion **sin sesiones (stateless)**: el servidor NO recuerda quien esta conectado; cada peticion del navegador lleva el token y el servidor lo verifica al instante.

Un JWT tiene 3 partes separadas por punto:

HEADER	PAYLOAD	SIGNATURE
Tipo de token y algoritmo (HS256)	Datos del usuario (username, roles, fecha expiracion)	Firma HMAC-SHA256 que garantiza autenticidad

2. La libreria JWT: donde se declara

La libreria usada en el **Backend (Java/Spring Boot)** se llama **jjwt** (Java JWT) de io.jsonwebtoken, version 0.11.5. Se declara en el archivo Maven:

Archivo: **BackEnd/pom.xml**

Lineas	Artefacto	Para que sirve
44-47	jjwt-api	La interfaz publica: las clases que usamos en el codigo (Jwt, Claims, etc.)
49-52	jjwt-impl	La implementacion interna. Solo actua en tiempo de ejecucion (scope runtime)
55-58	jjwt-jackson	Serializa el payload JSON del token. Tambien es solo runtime

Nota: el Frontend (JavaScript) NO usa ninguna libreria JWT. Solo guarda el token y lo decodifica manualmente con `atob()` para leer la fecha de expiracion. Toda la firma y verificacion vive en el backend.

3. Implementacion Backend (Java / Spring Boot)

El backend divide las responsabilidades JWT en archivos separados. Cada uno cumple una funcion clara:

3.1 JwtTokenProvider.java — El motor que crea y valida tokens

Ruta: `BackEnd/src/main/java/com/worldcup/predictor/config/JwtTokenProvider.java`

Lineas	Que hace
3-4	Importa Jwt, Claims, JwtException y Keys de io.jsonwebtoken (la libreria jjwt)
19-26	@Value lee jwt.secret y jwt.expiration-ms desde application.properties
30-33	@PostConstruct: convierte el texto secreto en clave HMAC-SHA256 con Keys.hmacShaKeyFor()
35-38	generateAccessToken(): genera el token de acceso de corta duracion (ej 15 minutos)

40-43	generateRefreshToken(): genera el token de refresco de larga duracion (ej 7 dias)
53-70	buildToken() — construye el JWT real: setSubject (username), setIssuedAt, setExpiration, claim('roles'), signWith(clave, HS256), compact() devuelve el string final
72-78	getUsernameFromToken(): parsea el token y extrae el campo 'sub' (username del usuario)
81-88	validateToken(): verifica firma y expiracion. Devuelve true si es valido, false si no

3.2 JwtAuthenticationFilter.java — El guardian de cada peticion HTTP

Ruta: BackEnd/src/main/java/com/worldcup/predictor/config/JwtAuthenticationFilter.java

Lineas	Que hace
19	Extiende OncePerRequestFilter: Spring garantiza que se ejecute UNA sola vez por request
30-44	doFilterInternal(): el metodo principal. Extrae el token del header, llama a validateToken(), carga el usuario de la BD y lo registra en el SecurityContext de Spring. Eso hace que Spring sepa quien es el usuario para ese request
49-55	getJwtFromRequest(): busca el header 'Authorization'. Si empieza con 'Bearer ' extrae el token cortando los primeros 7 caracteres del string

3.3 SecurityConfig.java — Las reglas de seguridad

Ruta: BackEnd/src/main/java/com/worldcup/predictor/config/SecurityConfig.java

Lineas	Que hace
37	SessionCreationPolicy.STATELESS: Spring NO crea sesiones HTTP. Cada request es independiente y debe autenticarse con su token
39-41	Rutas publicas: /api/auth/** (login y registro) no necesitan token. Cualquier otra ruta requiere autenticacion
43	addFilterBefore(): registra JwtAuthenticationFilter para que se ejecute ANTES que el filtro de login clasico de Spring
49-51	BCryptPasswordEncoder: las contraseñas se guardan hasheadas en BD, nunca en texto plano
67-77	CORS: permite que el frontend en puerto 8000 llame al backend en 8080 sin que el navegador lo bloquee. Habilita el header Authorization

3.4 AuthController.java — Los endpoints de login y registro

Ruta: BackEnd/src/main/java/com/worldcup/predictor/controller/AuthController.java

Lineas	Que hace
39-61	/api/auth/login (POST): recibe username y password, los verifica con AuthenticationManager contra la BD, y si son correctos genera accessToken y refreshToken con JwtTokenProvider. Los devuelve en JSON al frontend
63-67	/api/auth/register (POST): guarda el nuevo usuario en BD con contraseña BCrypt
69-87	/api/auth/refresh (POST): recibe el refreshToken, lo valida con JwtTokenProvider, y genera tokens nuevos. Sirve para renovar sesion sin pedir login de nuevo
89-257	Clases internas (LoginRequest, RegisterRequest, AuthResponse...): contenedores de datos para estructurar el JSON de entrada y salida

3.5 CustomUserDetails.java y CustomUserDetailsService.java

Ruta: `BackEnd/src/main/java/com/worldcup/predictor/config/`

CustomUserDetails.java (lineas 11-62): envuelve nuestro User de la base de datos en el formato que Spring Security entiende (interfaz UserDetails). Implementa metodos como getUsername(), getPassword(), getAuthorities() (roles). Sin esta clase Spring no sabe como leer nuestro usuario propio.

CustomUserDetailsService.java (lineas 20-24): cuando el filtro JWT necesita cargar un usuario por su username, este servicio busca en la BD con UserRepository.findByUsername() y devuelve un CustomUserDetails.

4. Implementacion Frontend (JavaScript)

El frontend no usa ninguna libreria JWT. Trata el token como un string: lo guarda en localStorage, lo envia en cada peticion HTTP, y cuando necesita saber si expiro lo decodifica manualmente con JavaScript puro usando atob().

4.1 js/config.js — Configuracion central

Lineas	Que hace
9-17	API_BASE_URL: construye la URL del backend usando el hostname del navegador. Esto permite que funcione tanto en localhost como en red local (LAN)
21-38	ENDPOINTS: define todas las rutas de la API incluyendo AUTH.LOGIN (/auth/login), AUTH.REGISTER (/auth/register) y AUTH.REFRESH (/auth/refresh)
41-46	STORAGE: nombres de claves de localStorage: 'jwt_token' para el accessToken y 'refresh_token' para el refreshToken

4.2 js/auth.js — El gestor de autenticacion del navegador

Ruta: `FrontEnd/js/auth.js` — Este es el archivo mas importante del JWT en el frontend.

Lineas	Que hace
6-18	Constructor: al cargar la pagina recupera token, refreshToken y datos del usuario desde localStorage. Si hay token pero no datos, los extrae del propio token con getUserFromToken()
26-58	login(): POST a /api/auth/login. Si el servidor responde OK, llama a setTokens() y setUser() para persistir todo en localStorage
67-89	register(): POST a /api/auth/register. No guarda token (el usuario debe hacer login)
94-102	logout(): elimina 'jwt_token', 'refresh_token' y 'user_data' de localStorage
108-110	isAuthenticated(): devuelve true solo si existe token Y no esta expirado
125-132	setTokens(): persiste accessToken y refreshToken en localStorage
181-195	decodeToken(): AQUI se decodifica el JWT sin libreria. Toma la parte central (payload), la decodifica con atob() y la parsea como JSON
201-205	isTokenExpired(): lee el campo 'exp' del payload (segundos Unix) y compara con Date.now() en milisegundos. Si $exp * 1000 < \text{ahora}$, el token expiro
211-222	getUserFromToken(): extrae username, name y roles del payload del token
224-252	refreshAccessToken(): POST a /api/auth/refresh con el refreshToken para obtener nuevos tokens sin pedir login. Si falla hace logout automatico

4.3 js/api.js — Como se envia el token en cada peticion

Lineas	Que hace
26-28	Si el usuario esta autenticado, agrega el header: Authorization: Bearer <token> Este es el header que JwtAuthenticationFilter lee en el backend
46-54	Si el servidor devuelve 401 (no autorizado), intenta refreshAccessToken() automaticamente. Si funciona, reintenta la peticion original sin que el usuario note nada. Si el refresh falla, redirige al login

4.4 Paginas HTML — Donde se usa el token en la interfaz de usuario

Archivo	Linea	Que hace con JWT
html/login.html	163	Llama a authManager.login() al enviar el formulario de login
html/login.html	190	Al cargar: si ya hay token valido redirige al dashboard directamente
html/dashboard.html	226	Al cargar: verifica isAuthenticated() y si el token expiro
html/dashboard.html	231	Si el token expiro intenta refreshAccessToken(). Si falla, manda al login
html/dashboard.html	239	Lee nombre del usuario con authManager.getUser() y lo muestra en la barra
html/dashboard.html	452	Boton logout: llama authManager.logout() que limpia localStorage

5. Flujo completo de autenticacion paso a paso

1 LOGIN	El usuario escribe usuario y contrasena en login.html (linea 163) y llama a authManager.login() en auth.js (linea 26). Este hace POST a /api/auth/login. AuthController.java (linea 42) verifica credenciales, llama a JwtTokenProvider.generateAccessToken() (linea 35) y devuelve los tokens.
2 GUARDADO	auth.js linea 125 (setTokens()) guarda el accessToken en localStorage como 'jwt_token' y el refreshToken como 'refresh_token'. Tambien guarda nombre y roles del usuario.
3 PETICION PROTEGIDA	Cuando el dashboard pide datos, api.js linea 27 agrega el header: Authorization: Bearer eyJhbGci... JwtAuthenticationFilter (linea 33) extrae el token, llama a validateToken() y si es valido registra al usuario en el SecurityContext de Spring.
4 TOKEN EXPIRADO	Si el backend devuelve 401, api.js linea 47 llama a authManager.refreshAccessToken() (auth.js linea 224). Este hace POST a /api/auth/refresh con el refreshToken. El backend genera tokens nuevos y el usuario ni se entera.
5 LOGOUT	El usuario hace clic en 'Cerrar Sesion' (dashboard.html linea 452). authManager.logout() (auth.js linea 94) elimina los tres items de localStorage. El backend no hace nada: al ser stateless no tiene sesion que cerrar.

6. Mapa de todos los archivos JWT de un vistazo

ARCHIVO	LADO	ROL EN JWT
BackEnd/pom.xml	Backend	Declara la libreria jjwt 0.11.5 (lineas 44-58)

config/JwtTokenProvider.java	Backend	Crea y valida tokens con la libreria jjwt
config/JwtAuthenticationFilter.java	Backend	Intercepta cada HTTP request y verifica el token
config/SecurityConfig.java	Backend	Reglas de seguridad: rutas publicas, stateless, CORS
config/CustomUserDetailsService.java	Backend	Carga el usuario de la BD cuando el filtro lo necesita
config/CustomUserDetails.java	Backend	Adapta nuestro User a la interfaz de Spring Security
controller/AuthController.java	Backend	Endpoints /login, /register, /refresh que generan tokens
FrontEnd/js/config.js	Frontend	Define URL de la API y claves de localStorage
FrontEnd/js/auth.js	Frontend	Guarda, lee, decodifica y renueva el token JWT
FrontEnd/js/api.js	Frontend	Envia el token en cada peticion como header Bearer
FrontEnd/html/login.html	Frontend	Inicia el flujo (llama a authManager.login)
FrontEnd/html/dashboard.html	Frontend	Verifica token al cargar; muestra datos del usuario